Algorithmic Cost of Voronoi Diagrams

1. a Sweep Line Algorithm
   - complexity and characterization
   - running an example of CGAL
   - a sweep line algorithm
2. the Algorithmic Cost
   - justification for $O(n\log(n))$
   - degenerate cases
3. Applications
   - Voronoi diagrams of line segments
   - farthest-point Voronoi diagrams

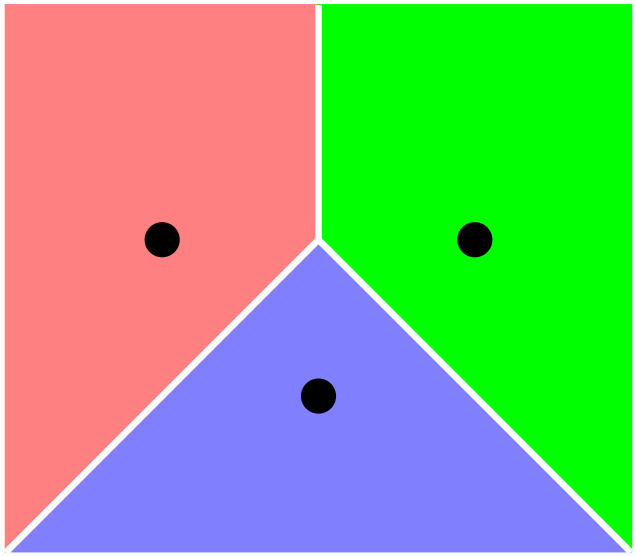
MCS 481 Lecture 23
Computational Geometry
Jan Verschelde, 16 October 2026

Algorithmic Cost of Voronoi Diagrams

1. a Sweep Line Algorithm
   - complexity and characterization
   - running an example of CGAL
   - a sweep line algorithm
2. the Algorithmic Cost
   - justification for $O(n \log(n))$
   - degenerate cases
3. Applications
   - Voronoi diagrams of line segments
   - farthest-point Voronoi diagrams

complexity of the Voronoi diagram

For the post office problem, a Voronoi diagram partitions the plane in convex regions of all points closest to the same post office.

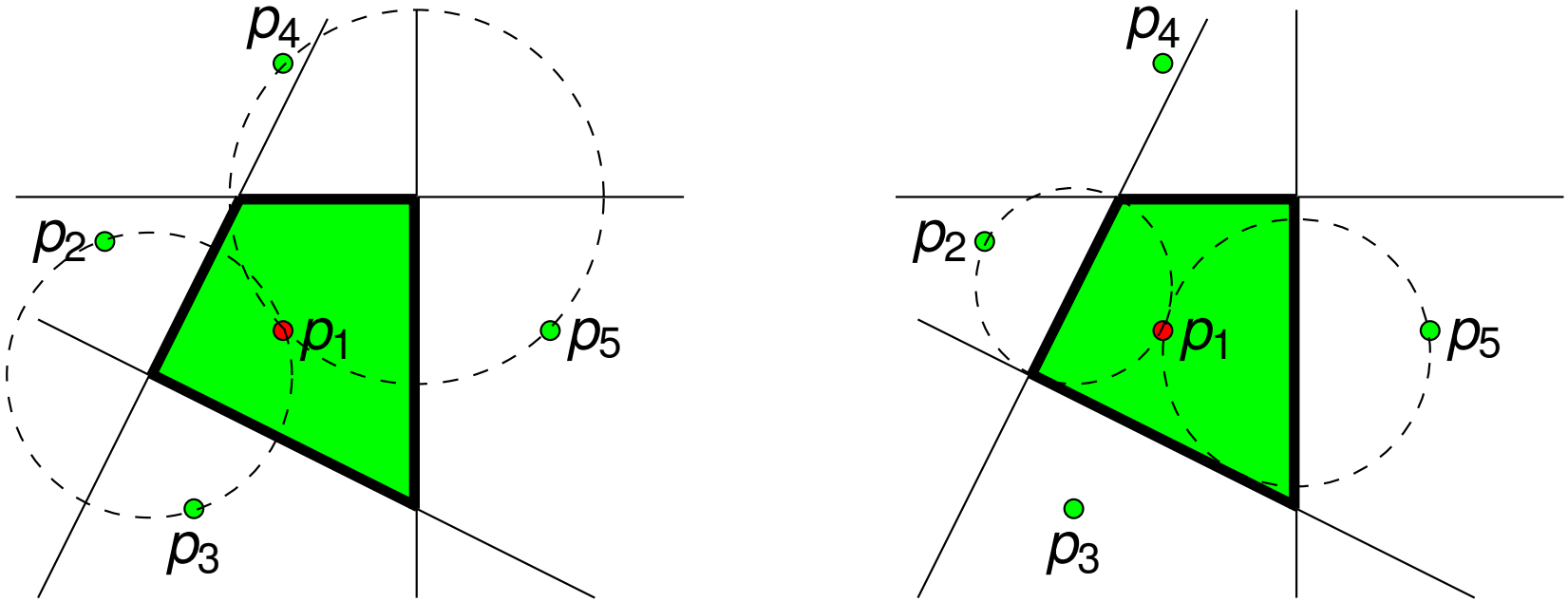


Theorem (complexity of a Voronoi diagram)

Let $\mathrm{Vor}(P)$ *be a Voronoi diagram for* n *points,* $n \geq 3$.

1. *The number of vertices of* $\mathrm{Vor}(P)$ *is at most* $2n - 5$.
2. *The number of edges of* $\mathrm{Vor}(P)$ *is at most* $3n - 6$.

characterization of vertices and edges



Theorem (characterization of vertices and edges)

Let $\mathrm{Vor}(P)$ *be the Voronoi diagram of the set* P*.*

1. q *is a vertex of* $\mathrm{Vor}(P)$
   $\Leftrightarrow$ $C_P(q)$ *contains 3 or more points of* P *on its boundary.*
2. e *is an edge of* $\mathrm{Vor}(P) \Leftrightarrow$ e *is on the bisector between* p_i *and* p_j *and there is a* $q \in e$ *so* $C_P(q)$ *has only* p_i *and* p_j *on its boundary.*

Algorithmic Cost of Voronoi Diagrams

1. a Sweep Line Algorithm
   - complexity and characterization
   - running an example of CGAL
   - a sweep line algorithm
2. the Algorithmic Cost
   - justification for $O(n \log(n))$
   - degenerate cases
3. Applications
   - Voronoi diagrams of line segments
   - farthest-point Voronoi diagrams

running an example of CGAL

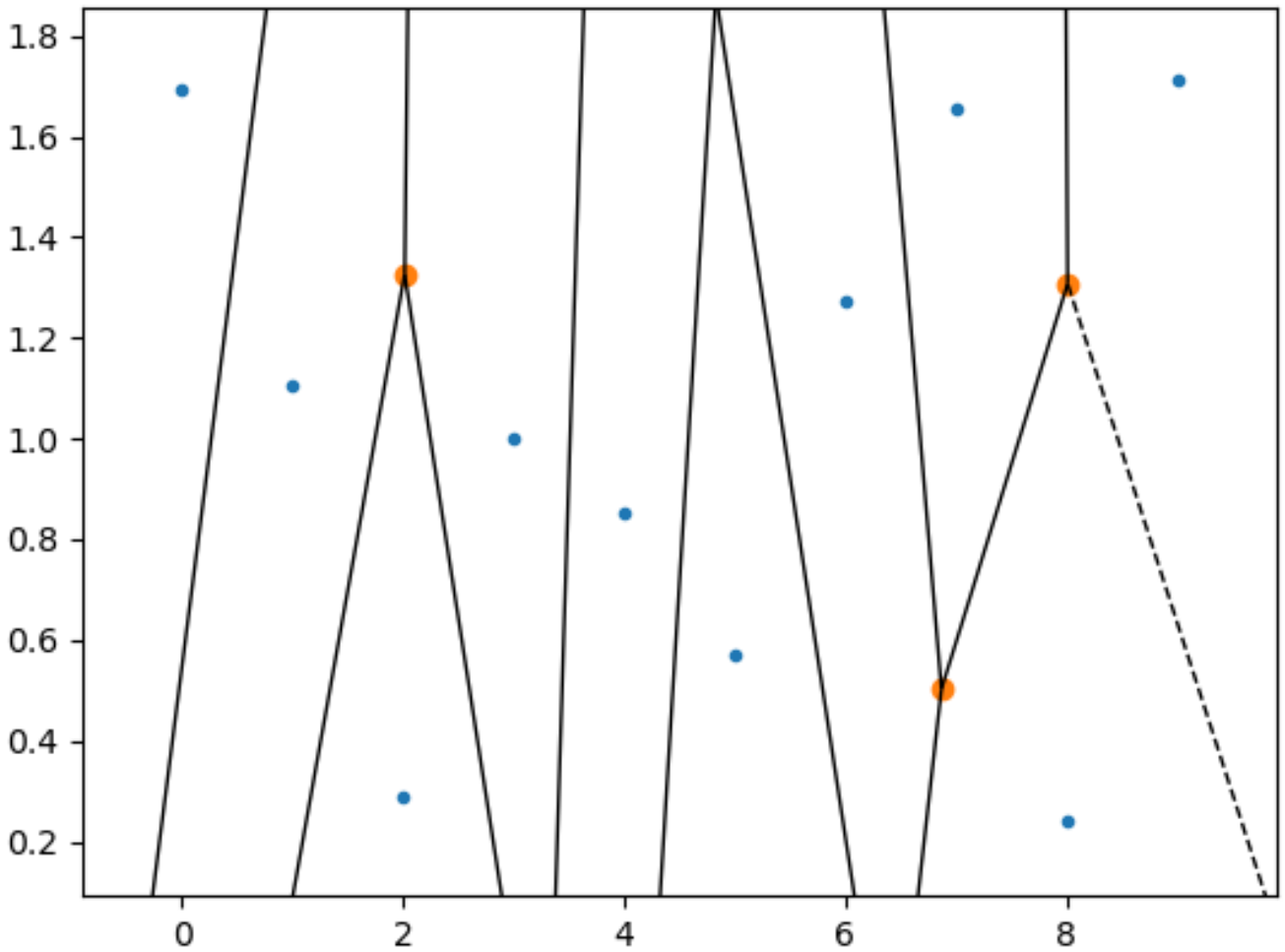
On a Window Subsystem for Linux (WSL), running Ubuntu:

1. Run `sudo apt-get install` followed by `libgmp-dev`, `libmpfr-dev`, `libcgal-dev`.
2. Download `CGAL-5.6-examples.zip`.
3. Compile `vd_2_point_location.cpp` from the `Voronoi_diagram_2` folder.
4. Before running, copy the input data.

The Python bindings (see the previous lecture) follow the same logic as the C++ code.

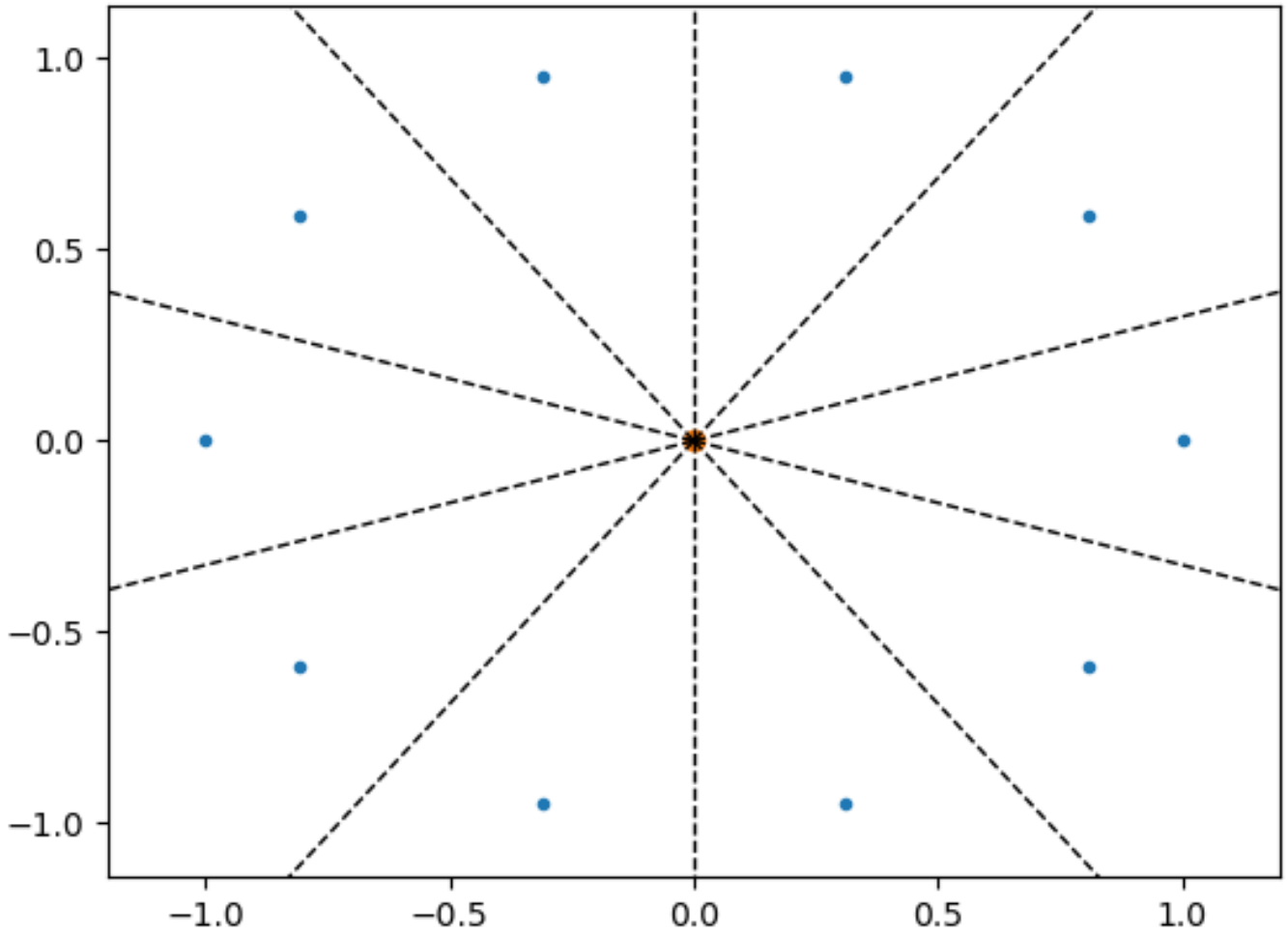
the bounding box

An example made with `voronoi_plot_2d` of `scipy.spatial`:



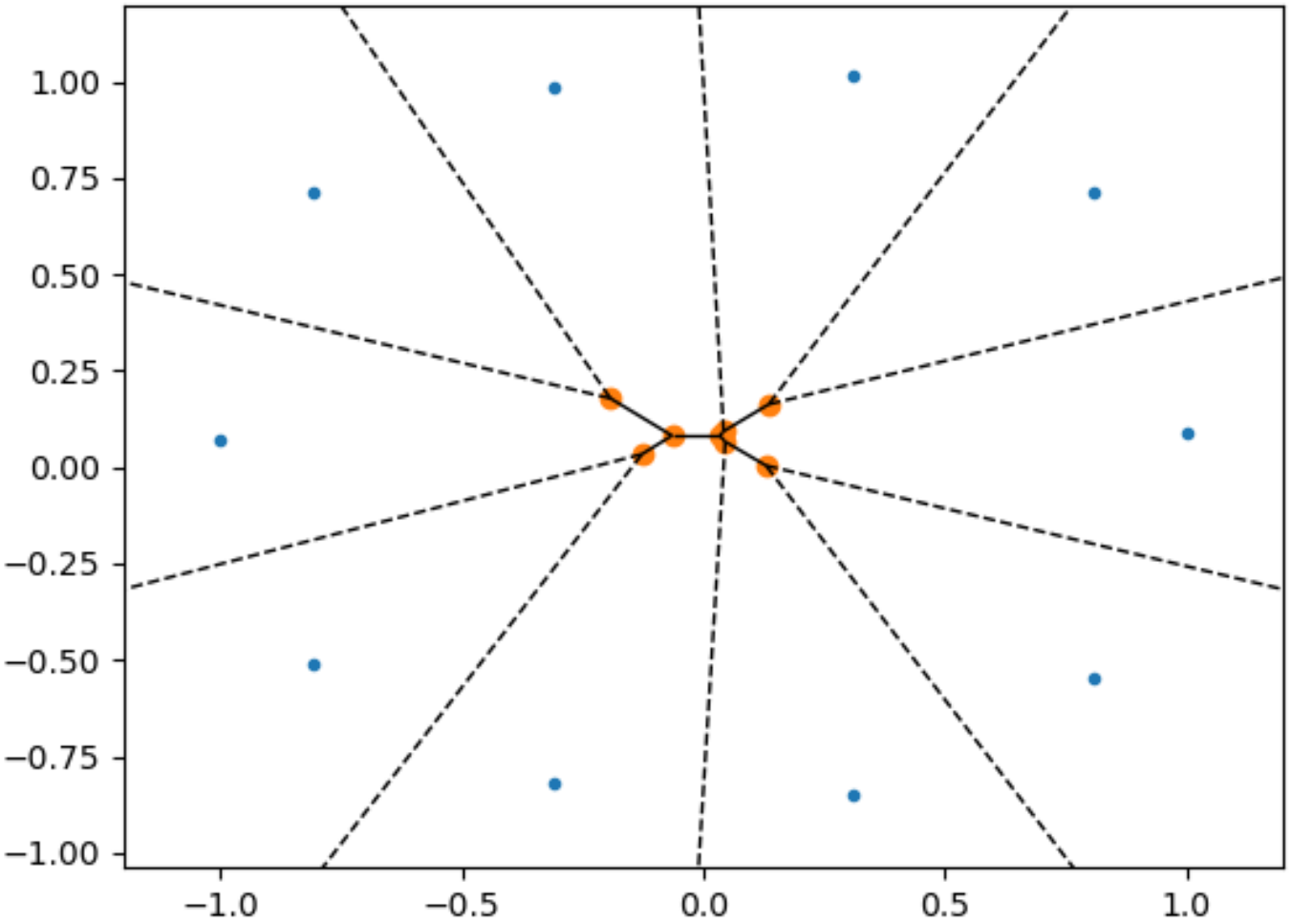
a vertex of high degree

An example made with `voronoi_plot_2d` of `scipy.spatial`:



a cluster of vertices

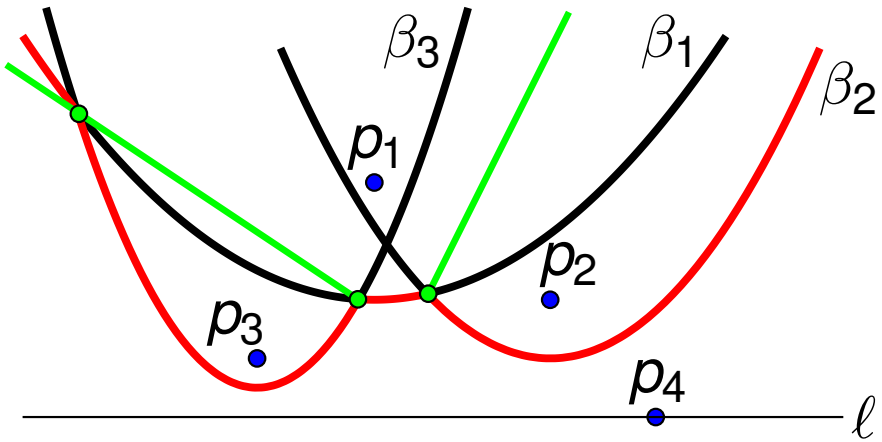
An example made with `voronoi_plot_2d` of `scipy.spatial`:

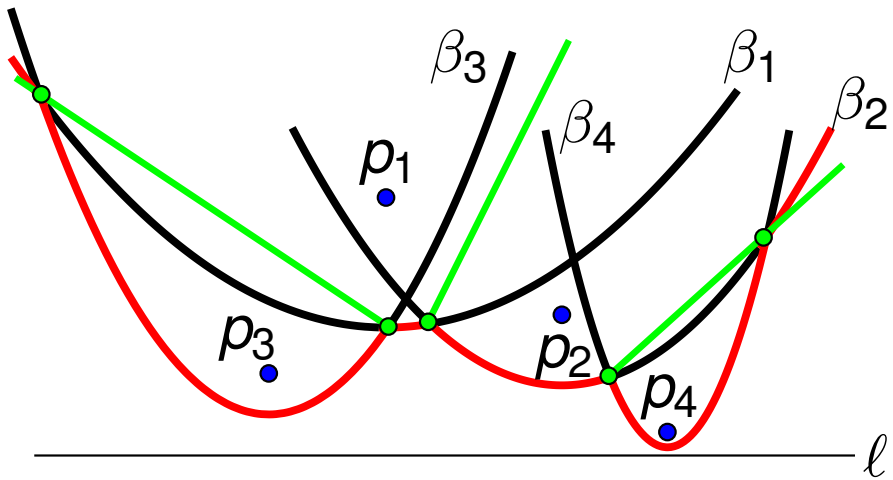


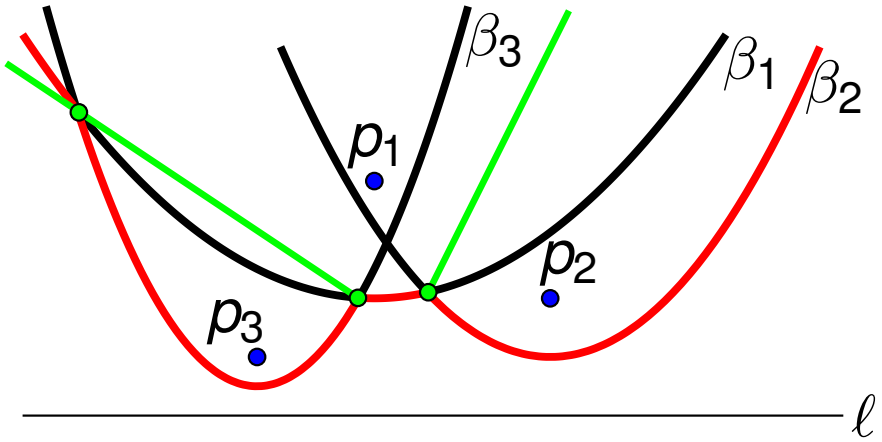
Algorithmic Cost of Voronoi Diagrams

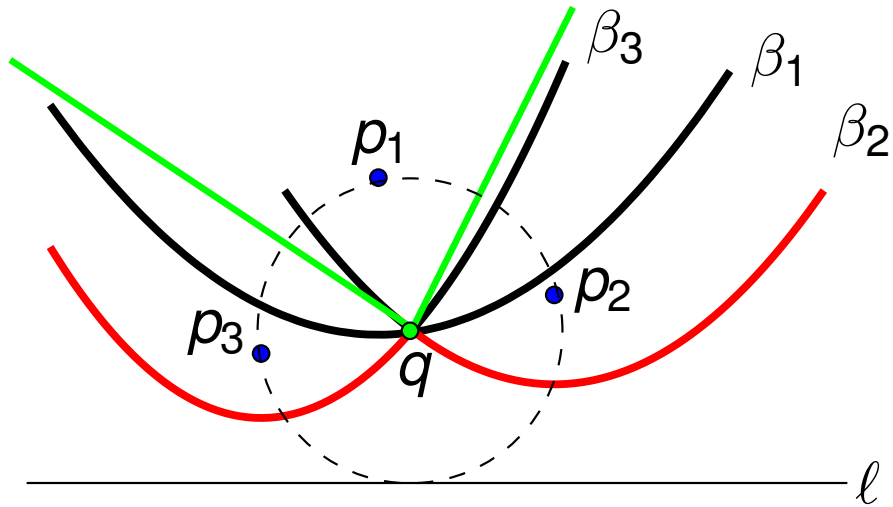
1. a Sweep Line Algorithm
   - complexity and characterization
   - running an example of CGAL
   - a sweep line algorithm
2. the Algorithmic Cost
   - justification for $O(n \log(n))$
   - degenerate cases
3. Applications
   - Voronoi diagrams of line segments
   - farthest-point Voronoi diagrams

site and circle events change the parabolic arcs









Exercise 1: Draw the above pictures with the Manhattan metric
$\text{dist}_1(p, q) = |p_x - q_x| + |p_y - q_y|$, $p_1 = (0, 10)$, $p_2 = (7.5, 5)$, $p_3 = (-5, 2.5)$.

the beach line and the Voronoi diagram

Definition (the beach line)

The sequence of parabolic arcs that separates the points closer to any site than to ℓ is *the beach line*.

Everything of Vor(P) above the beach line remains as ℓ moves down.

Lemma (new parabolic arc through site event)

The only way in which a new parabolic arc can appear on the beach line is through a site event.

Lemma (disappearing parabolic arc through circle event)

The only way in which an existing arc can disappear from the beach line is through a circle event.

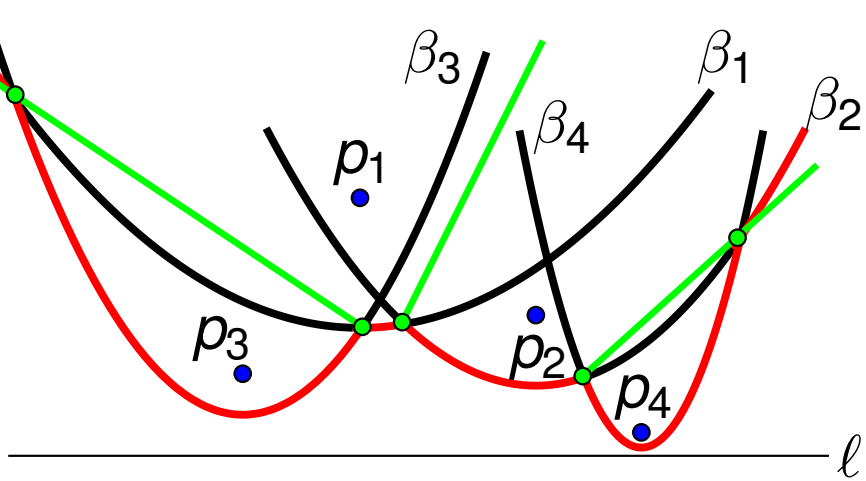
Every Voronoi vertex is detected by a circle event.

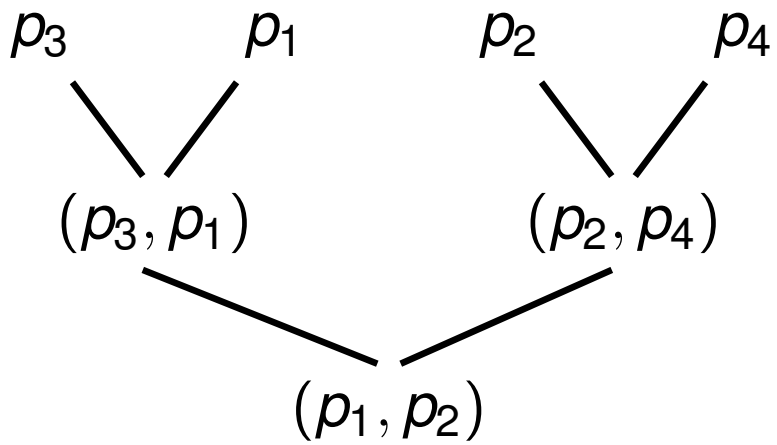
a balanced binary search tree $\mathcal{T}$

The beach line is represented by a balanced binary search tree $\mathcal{T}$:

- adjacent leaves are adjacent arcs; and
- internal nodes just above arcs store breakpoints between arcs.

For example:





formulation of the algorithm

Algorithm VORONOIDIAGRAM(P)

Input: n sites in the set P, sorted on height.
Output: Vor(P) stored in a doubly connected edge list $\mathcal{D}$.

1. Initialize $\mathcal{Q}$ with P; $\mathcal{T} = \emptyset$; $\mathcal{D} = \emptyset$
2. while $\mathcal{Q} \neq \emptyset$ do
3. pop next event e from $\mathcal{Q}$
4. if e is a site event
5. then HANDLESITEEVENT(e), e is a site p_i
6. else HANDLECIRCLEEVENT(γ), γ is leaf of $\mathcal{T}$
7. compute bounding box R for half line edges
8. update $\mathcal{D}$ with R

handling a site event

Algorithm HANDLESITEEVENT($\mathcal{Q}, \mathcal{T}, \mathcal{D}, p_i$)

Input: event queue $\mathcal{Q}$, tree $\mathcal{T}$, DCEL $\mathcal{D}$, site p_i.

1. if $\mathcal{T} = \emptyset$ then
2. insert p_i in $\mathcal{T}$

else

3. let α be the arc in $\mathcal{T}$ above p_i
4. if α has a pointer to a circle event e in $\mathcal{Q}$, then delete e from Q
5. replace leaf in $\mathcal{T}$ for α with subtree of 3 leaves
6. rebalance $\mathcal{T}$ if necessary
7. create new half edge records in $\mathcal{D}$ for edges separating cells
8. for all triple consecutive arcs that contain p_i as breakpoints
9. add circle events to $\mathcal{Q}$

handling a circle event

Algorithm HANDLECIRCLEEVENT($\mathcal{Q}, \mathcal{T}, \mathcal{D}, \gamma$)

Input: event queue $\mathcal{Q}$, tree $\mathcal{T}$, DCEL $\mathcal{D}$, leaf γ of $\mathcal{T}$.

1. update $\mathcal{T}$:
   1. delete leaf γ which represents the disappearing arc from $\mathcal{T}$
   2. update breakpoints at internal nodes, rebalance $\mathcal{T}$ if necessary
   3. delete circle events involving α
2. update $\mathcal{D}$:
   1. insert new vertex record for center q of $C_P(q)$
   2. create two half edge records for new breakpoints
   3. set pointers between half edges, attach new half edges at new q
3. update $\mathcal{Q}$:
   1. search for new circle events created by disappearing arc α
   2. insert these new circle events into $\mathcal{Q}$

Algorithmic Cost of Voronoi Diagrams

1. a Sweep Line Algorithm
   - complexity and characterization
   - running an example of CGAL
   - a sweep line algorithm
2. the Algorithmic Cost
   - justification for $O(n \log(n))$
   - degenerate cases
3. Applications
   - Voronoi diagrams of line segments
   - farthest-point Voronoi diagrams

running time of the sweep line algorithm

In the sweep line algorithm the points in P are sorted on height.

Lemma (cost of the sweep line algorithm)

The sweep line algorithm for n sites
takes $O(n \log(n))$ operations and $O(n)$ storage.

The data structures consist of

1. the event queue $\mathcal{Q}$,
2. a balanced binary search tree $\mathcal{T}$, and
3. the doubly connected edge list $\mathcal{D}$ to store Vor(P).

algorithmic cost for $\mathcal{D}$ and $\mathcal{T}$

- The beach line is represented by a balanced binary search tree $\mathcal{T}$.

  The number of leaves in $\mathcal{T}$ is bounded by the number of arcs on the beach line, which is bounded by $2n - 1$.

  As $\mathcal{T}$ is balanced, searching and manipulating $\mathcal{T}$ runs in $O(\log(n))$ time and takes $O(n)$ storage.

- Vor(P) is stored in a doubly connected edge list $\mathcal{D}$ and the primitive operations on $\mathcal{D}$ take $O(1)$ time.

  The number of vertices and edges in Vor(P) is $O(n)$, so storage is $O(n)$.

algorithmic cost for $\mathcal{Q}$

- The size of the event queue $\mathcal{Q}$ is $O(n)$ because
  1. the number of site events is bounded by the number of sites,
  2. the number of circle events is bounded by the number of vertices,

  both of which are $O(n)$.

- But some triples of three consecutive arcs on the beach line may never lead to a circle event and yet be stored on $\mathcal{Q}$.

  To prevent storing too many triples as events in $\mathcal{Q}$, the algorithm checks whether there can be an intersection point of the two bisectors between two consecutive arcs below the sweep line, before adding the event to $\mathcal{Q}$.

Exercise 2: Explain how such check might work.

summary of the cost

As the number of events is thus $O(n)$ and the cost for handling an event is $O(\log(n))$, we arrive at $O(n \log(n))$ operations.

Exercise 3: Consider the pseudo code for the subroutines HANDLESITEEVENT and HANDLECIRCLEEVENT.

Justify the cost of those subroutines referring to their codes.

Algorithmic Cost of Voronoi Diagrams

1. a Sweep Line Algorithm
   - complexity and characterization
   - running an example of CGAL
   - a sweep line algorithm
2. the Algorithmic Cost
   - justification for $O(n \log(n))$
   - degenerate cases
3. Applications
   - Voronoi diagrams of line segments
   - farthest-point Voronoi diagrams

degenerate cases

0. Detect if points are collinear in the preprocessing stage.
1. Points with the same y-coordinate.
   - Break ties with the x-coordinate, e.g.: leftmost comes first.
   - Special code is needed to handle the start of the algorithm.
2. Vertices of Vor(P) of degree 4 or higher.
   - The algorithm gives vertices of degree 3 at the same location.
   - There will be edges of length zero.

   Degenerate edges can be removed and vertices with the same coordinates joined in a post processing stage.

Theorem (algorithmic cost of a Voronoi diagram)

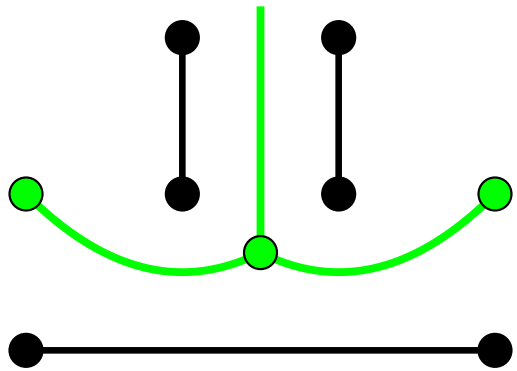
Computing a Voronoi diagram for n sites takes $O(n \log(n))$ operations and $O(n)$ storage.

Algorithmic Cost of Voronoi Diagrams

1. a Sweep Line Algorithm
   - complexity and characterization
   - running an example of CGAL
   - a sweep line algorithm
2. the Algorithmic Cost
   - justification for $O(n \log(n))$
   - degenerate cases
3. Applications
   - Voronoi diagrams of line segments
   - farthest-point Voronoi diagrams

Voronoi diagrams of line segments

Given a set of line segments,
partition the points in the plane in regions closest to a line segment.



Edges of the diagram are parabolic arcs if the end point of one line segment is the closest point to another line segment.

Robot motion planning

Line segments represent obstacles a robot has to avoid.

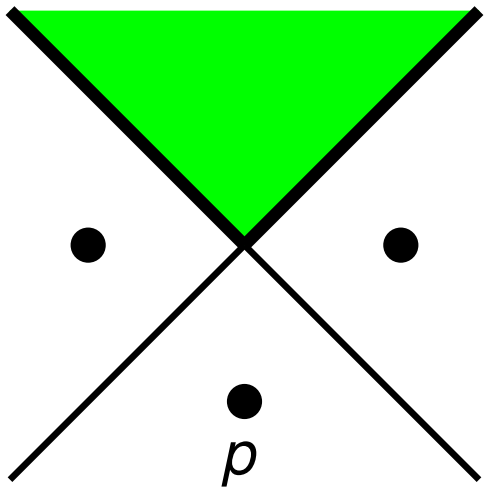
Algorithmic Cost of Voronoi Diagrams

1. a Sweep Line Algorithm
   - complexity and characterization
   - running an example of CGAL
   - a sweep line algorithm
2. the Algorithmic Cost
   - justification for $O(n \log(n))$
   - degenerate cases
3. Applications
   - Voronoi diagrams of line segments
   - farthest-point Voronoi diagrams

farthest-point Voronoi diagrams

The farthest-point Voronoi diagram divides the plane into cells in which the same point is the farthest point.

A cell in the farthest-point Voronoi diagram is the intersection of the half planes defined by bisectors between the points, taking the half of the plane farthest from the point in the cell.



Roundness testing of manufactured objects (metrology)

Find smallest-width annulus which contains all sampled points.

summary and exercises

Section 7.1 and 7.2 in the textbook were covered, with a glimpse of sections 7.3 and 7.4.

Consider the following activities, listed below.

1. Write the solutions to exercises 1, 2, and 3.
2. Consult the CGAL documentation and example code on Voronoi diagrams.
3. Consider the exercises 9, 10, 11 in the textbook.
4. Explore the possibilities for a final project.